

**SRI CHANDRASEKHARENDRA SARASWATHI VISWA
MAHAVIDYALAYA**

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM - 631561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



Name : B. Navatha . Reddy

Reg. No : 112U9A045

Class : II B.E. (CSE)

Course Code: BCSF244P90

Course Name: Operating system lab.

SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM - 631561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



BONAFIDE CERTIFICATE

This is to certify that this is the bonafide record of work done by
Mr/Ms. B. Navatha seddy

with Reg. No 11249A045 of II-B.E.(CSE) during the
academic year 2025 - 2026.

operating system lab
BCSF244P90

Station: Enathur

Date: 25/4/26.

[Signature]
Staff-in-charge 25/4/26

[Signature]

Head of the Department

Submitted for the Practical examination held on _____.

Examiner-1

Examiner-2

INDEX

SL.NO	ProgramName	Date	Page no	Signature
1	Linux Commands	3/2/26.		}
2	Grep Commands	3/2/26.		
3	Awk Command linux	17/3/26.		
4a	System calls of UNIX			}
	operating system	10/3/26.		
4b	System calls of UNIX			
	operating system (wait)	10/3/26		}
4c	Getpid()	10/3/26		
4d	opendir, Readdir	10/3/26.		
4e	System calls of unix			}
	operating system	10/3/26		
4f	open, read & write	10/3/26.		
5a	Process management (fork)	17/3/26		}
5b	Exec()	17/3/26		
6a	FIFO cpu scheduling	31/3/26.		

[illegible]

Ex 1 - UNIX Commands

1. pwd

```
ghost@LAPTOP-HNSF11IH:~$ pwd
/home/ghost
```

2. ls

```
ghost@LAPTOP-HNSF11IH:/mnt$ ls
ws1  wslg
```

3. cd

```
ghost@LAPTOP-HNSF11IH:/$ cd mnt/
ghost@LAPTOP-HNSF11IH:/mnt$
```

4. mkdir

```
ghost@LAPTOP-HNSF11IH:~$ mkdir Temp
ghost@LAPTOP-HNSF11IH:~$ ls
Temp
```

5. rmdir

```
ghost@LAPTOP-HNSF11IH:~$ rmdir Temp
ghost@LAPTOP-HNSF11IH:~$ ls
ghost@LAPTOP-HNSF11IH:~$
```

6. cat

```
ghost@LAPTOP-HNSF11IH:~$ cat sample.txt
Hello World!
```

7. mv

```
ghost@LAPTOP-HNSF11IH:~$ mv sample.txt Temp/
ghost@LAPTOP-HNSF11IH:~$ ls
Temp
ghost@LAPTOP-HNSF11IH:~$ cd Temp/
ghost@LAPTOP-HNSF11IH:~/Temp$ ls
sample.txt
```

8. rm

```
ghost@LAPTOP-HNSF11IH:~/Temp$ ls
sample.txt
ghost@LAPTOP-HNSF11IH:~/Temp$ rm sample.txt
ghost@LAPTOP-HNSF11IH:~/Temp$ ls
ghost@LAPTOP-HNSF11IH:~/Temp$
```

9. ps

```
ghost@LAPTOP-HNSF11IH:~/Temp$ ps
  PID TTY          TIME CMD
  183 pts/0        00:00:00 bash
  240 pts/0        00:00:00 ps
```

10. who

```
ghost@LAPTOP-HNSF11IH:~/Temp$ who
ghost    pts/1        2026-04-21 11:49
```

11. whoami

```
ghost@LAPTOP-HNSF11IH:~/Temp$ whoami
ghost
```

12. date

```
ghost@LAPTOP-HNSF11IH:~/Temp$ date
Tue Apr 21 12:06:44 PM UTC 2026
```

13. echo

```
ghost@LAPTOP-HNSF11IH:~/Temp$ echo Hello
Hello
```

14. head

```
ghost@LAPTOP-HNSF11IH:/mnt/c/Users/aksha/Downloads$ head sample.txt
1
2
3
4
5
6
7
8
9
10
```

15. tail

```
ghost@LAPTOP-HNSF11IH:/mnt/c/Users/aksha/Downloads$ tail sample.txt
11
12
13
14
15
16
17
18
19
20ghost@LAPTOP-HNSF11IH:/mnt/c/Users/aksha/Downloads$
```


16. wc

```
ghost@LAPTOP-HNSF11IH:/mnt/c/Users/aksha/Downloads$ wc sample.txt
19 20 69 sample.txt
```

17. find

```
ghost@LAPTOP-HNSF11IH:/mnt/c/Users/aksha/Downloads$ find sample.txt
sample.txt
```

Ex 2 - GREP Commands

-e	Use extended regular expressions as search
-f	Specify a file which has content to be searched
-i	Search for string as case insensitive
-w	Search given string as a whole word
-l	Display files that contain the specified string out of all the specified files
-o	Print only the word which has specified string
-v	Show lines which doesn't contain specified string
-c	Show no. of lines which contains specified string

1. -e

```
ghost@LAPTOP-HNSF11IH:~$ grep "th" a.txt
this is Bash
and as your already know, thIS IS alSO baSH
```

2. -i

```
ghost@LAPTOP-HNSF11IH:~$ grep -i "BASH" a.txt
this is Bash
This is also bash
and THis is aLSO bASh
and as your already know, thIS IS alSO baSH
```

3. -l

```
ghost@LAPTOP-HNSF11IH:~$ grep -l "bash" a.txt b.txt
a.txt
```

4. -v

```
ghost@LAPTOP-HNSF11IH:~$ grep -v -i "bash" a.txt
Hello
world
```

5. -c

```
ghost@LAPTOP-HNSF11IH:~$ grep -c -i "bash" a.txt
4
```

Ex 3 - AWK Commands

Syntax: awk [options] [pattern] [file]

-F	Tells awk to use the specified string as field separator
\$n	Prints elements in N'th Column
-f	Dumps data to file
-v	Variable Condition

Patterns

NR	No. of records
NF	No. of fields
FS	Field Separator
RS	Record Separator
OFS	Output Field Separator
ORS	Output Record Separator

1. \$n

```
ghost@LAPTOP-HNSF11IH:~$ awk '{print$1}' a.csv
eid,
001,
002,
003,

ghost@LAPTOP-HNSF11IH:~$ awk '{print$3}' a.csv
exp
5
8
7
```

2. -F

```
ghost@LAPTOP-HNSF11IH:~$ awk -F ',' '{print$1}' a.csv
eid
001
002
003
```


3. NR

```
ghost@LAPTOP-HNSF11IH:~$ awk -F ', ' '{print "Line",NR,": ", $1}' a.csv
Line 1 : eid
Line 2 : 001
Line 3 : 002
Line 4 : 003
```

4. NF

```
ghost@LAPTOP-HNSF11IH:~$ awk -F ', ' '{print "Field: ",NF," ", "Element: ",$NF}' a.csv
Field: 3 Element: exp
Field: 3 Element: 5
Field: 3 Element: 8
Field: 3 Element: 7
```

5. OFS

```
ghost@LAPTOP-HNSF11IH:~$ awk -F ', ' 'BEGIN {OFS=" | "} {print $1, $2}' a.csv
eid | name
001 | ak
002 | hi
003 | bh
```

6. ORS

```
ghost@LAPTOP-HNSF11IH:~$ awk -F ', ' 'BEGIN {ORS=" --- "} {print $1}' a.csv
eid --- 001 --- 002 --- 003 --- ghost@LAPTOP-HNSF11IH:~$ █
```

Ex 4a – System Calls of Unix Operating System (stat())

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>

int main() {
    char path[256];
    struct stat *nfile;

    nfile = (struct stat*) malloc(sizeof(struct stat));

    printf("Enter file path: ");
    scanf("%s", path);
    stat(path, nfile);

    printf("User ID: %d\n", nfile->st_uid);
    printf("Block Size: %d\n", nfile->st_blksize);
    printf("Last Access Time: %d\n", nfile->st_atime);
    printf("Last Modify Time: %d\n", nfile->st_mtime);
    printf("Production Mode: %d\n", nfile->st_mode);
    printf("File Size: %d Bytes\n", nfile->st_size);
    printf("No. of Links: %d\n", nfile->st_nlink);
}
```

Output:

```
Enter file path: 1.c
User ID: 1000
Block Size: 4096
Last Access Time: 1776796698
Last Modify Time: 1776796663
Production Mode: 33261
File Size: 641 Bytes
No. of Links: 1
```

Ex 4b – System Calls of Unix Operating System (wait())

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pid, status, exitch;

    if ((pid = fork()) == -1) {
        perror("error");
        exit(0);
    }

    if (pid == 0) {
        sleep(1);
        printf("child process\n");
        exit(0);
    } else {
        printf("parent process\n");

        if ((exitch = wait(&status)) == -1) {
            perror("during wait()");
            exit(0);
        }
        printf("\nparent exiting\n");
        exit(0);
    }
}
```

Output:

```
parent process  
child process  
  
parent exiting
```


Ex 4c – System Calls of Unix Operating System (getpid())

Program:

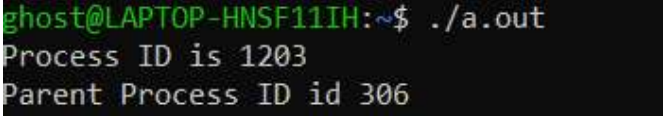
```
#include <stdio.h>

int main() {
    int pid;

    pid=getpid();
    printf("Process ID is %d\n",pid);

    pid=getppid();
    printf("Parent Process ID id %d\n",pid);
}
```

Output:

A terminal window with a black background and green text. The prompt is 'ghost@LAPTOP-HNSF11IH:~\$'. The command './a.out' has been executed, resulting in two lines of output: 'Process ID is 1203' and 'Parent Process ID id 306'.

```
ghost@LAPTOP-HNSF11IH:~$ ./a.out
Process ID is 1203
Parent Process ID id 306
```

Ex 4d – System Calls of Unix Operating System (opendir(), readdir())

Program:

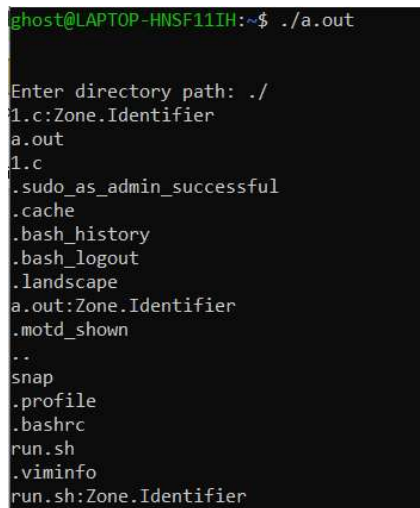
```
#include <stdio.h>
#include <dirent.h>

struct dirent *dptr;
int main(int argc, char *argv[]) {
    char buff[256]; DIR* dirp;

    printf("\n\nEnter directory path: ");
    scanf("%s", buff);

    if((dirp=opendir(buff)) == NULL) {
        printf("Error");
        exit(1);
    }
    while(dptr=readdir(dirp)) {
        printf("%s\n", dptr->d_name);
    }
    closedir(dirp);
}
```

Output:



```
ghost@LAPTOP-HNSF11IH:~$ ./a.out
Enter directory path: ./
l.c:Zone.Identifier
a.out
l.c
l.sudo_as_admin_successful
.cache
.bash_history
.bash_logout
.landscape
a.out:Zone.Identifier
.motd_shown
..
snap
.profile
.bashrc
run.sh
.viminfo
run.sh:Zone.Identifier
```

Ex 4e – System Calls of Unix Operating System (open())

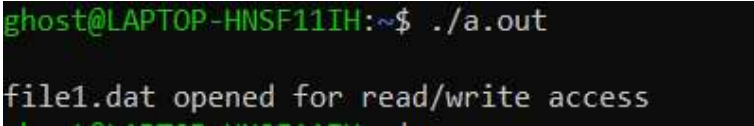
Program:

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = creat("file1.dat", S_IREAD | S_IWRITE);

    if(fd== -1) {
        printf("Error in opening file1.dat\n");
    } else {
        printf("\nfile1.dat opened for read/write access\n");
    }
    close(fd);
}
```

Output:



```
ghost@LAPTOP-HNSF11IH:~$ ./a.out
file1.dat opened for read/write access
```

Ex 4f – System Calls of Unix Operating System (open(), read(), write())

Program:

```
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>
#include <stdio.h>

static char message[] = "Hello world";

int main() {
    int fd = open("new2file.txt", O_RDWR | O_CREAT | O_EXCL, S_IREAD
| S_IWRITE);

    char buffer[80];

    if (fd != -1) {
        printf("new2file.txt opened for read/write access\n");
        write(fd, message, sizeof(message));
        lseek(fd, 0L, 0);

        if (read(fd, buffer, sizeof(message)) == sizeof(message))
            printf("\n\"%s\" was written to new2file.txt\n",
buffer);
        else
            printf("***Error reading new2file.txt***\n");
    } else {
        close(fd);
        printf("***new2file.txt already exists***\n");
    }
}
```

```
        return 0;  
    }
```

Output:

```
new2file.txt opened for read/write access  
"Hello world" was written to new2file.txt  
ghost@LAPTOP-HNSF11IH:~$ cat new2file.txt  
Hello worldghost@LAPTOP-HNSF11IH:~$ █
```


Ex 5a – Process Management (fork())

Program:

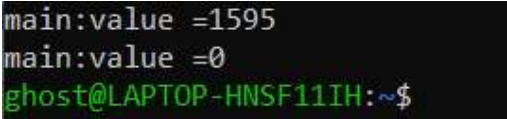
```
#include <stdio.h>

int main(void) {
    int fork(void), value;

    value=fork();
    printf("main:value =%d\n",value);

    return 0;
}
```

Output:

A terminal window with a black background and green text. It shows the output of the program: 'main:value =1595' on the first line, 'main:value =0' on the second line, and the shell prompt 'ghost@LAPTOP-HNSF11IH:~\$' on the third line.

```
main:value =1595
main:value =0
ghost@LAPTOP-HNSF11IH:~$
```

Ex 5b – Process Management (exec())

Program:

```
#include<stdio.h>

int main() {
    int pid;
    char *args[]={"/bin/ls","-l",0};

    printf("\nParent Process");
    pid=fork();

    if(pid==0) {
        execv("/bin/ls",args);
        printf("\nChild process");
    } else {
        wait();
        printf("\nParent process");
    }

    exit(0);
}
```

Output:

```
total 788
-rw-r--r-- 1 ghost ghost    274 Apr 22 04:16 1.c
-rw-r--r-- 1 ghost ghost     0 Apr 21 20:07 1.c:Zone.Identifier
-rwxr-xr-x 1 ghost ghost 790304 Apr 22 04:16 a.out
-rw-r--r-- 1 ghost ghost     0 Apr 21 20:07 a.out:Zone.Identifier
-rw-r--r-- 1 ghost ghost     0 Apr 22 03:52 file1.dat
-rw----- 1 ghost ghost    12 Apr 22 03:59 new2file.txt
-rw-r--r-- 1 ghost ghost    39 Apr 22 03:19 run.sh
-rw-r--r-- 1 ghost ghost     0 Apr 21 20:07 run.sh:Zone.Identifier
drwx----- 3 ghost ghost  4096 Apr 22 03:08 snap
Parent Process
Parent processghost@LAPTOP-HNSF11IH:~$
```

Ex 6a – FIFO (First In First Out) CPU Scheduling

Program:

```
#include <stdio.h>

struct process {
    int burst,wait;
}

p[20]={0,0};

int main() {
    int n,i,totalwait=0,totalturn=0;

    printf("\nEnter The No Of Process :");
    scanf ("%d",&n);

    for(i=0;i<n;i++) {
        printf("Enter The Burst Time (in ms) For Process # %2d\n",i+1);
        scanf ("%d",&p[i].burst);
    }

    printf("\nProcess \t Waiting Time TurnAround Time ");
    printf("\n \t (in ms) (in ms)");

    for(i=0;i<n;i++) {
        printf("\nProcess # %-12d%-15d%-15d",i+1,p[i].wait,p[i].wait+p[i].burst);
        p[i+1].wait=p[i].wait+p[i].burst;
        totalwait=totalwait+p[i].wait;
        totalturn=totalturn+p[i].wait+p[i].burst;
    }
}
```

```

    }

    printf("\n\nAVERAGE\n----- ");
    printf("\nWaiting Time : %f ms",totalwait/(float)n);
    printf("\nTurnAround Time : %f ms\n\n",totalturn/(float)n);

    return 0;
}

```

Output:

```

Enter The No Of Process :3
Enter The Burst Time (in ms) For Process # 1 :20
Enter The Burst Time (in ms) For Process # 2 :50
Enter The Burst Time (in ms) For Process # 3 :15

Process          Waiting Time TurnAround Time
      (in ms) (in ms)
Process # 1           0           20
Process # 2          20           70
Process # 3          70           85

AVERAGE
-----
Waiting Time : 30.000000 ms
TurnAround Time : 58.333332 ms

```

Ex 6b – Process Management (fork())

Program:

```
#include <stdio.h>
#include <stdlib.h>

int TRUE = 0;
int FALSE = -1;

int tbt[30], bt[30], tat[30], n = 0, wt[30];
int qt = 0, tqt = 0, time = 0, lmore, t_tat = 0, t_wt = 0;

void main() {
    int i, j;

    system("clear");

    printf("\nEnter no. of processors:");
    scanf("%d", &n);

    printf("\nEnter Quantum Time:");
    scanf("%d", &qt);

    for(i = 0; i < n; i++)
    {
        printf("\nEnter Burst Time of Processor[%d]:", i + 1);
        scanf("%d", &bt[i]);

        tbt[i] = bt[i];
        wt[i] = 0;
        tat[i] = 0;
    }
}
```

```
}
```

```
lmore = TRUE;
```

```
while(lmore == TRUE)
```

```
{
```

```
    lmore = FALSE;
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        if(bt[i] != 0)
```

```
            wt[i] = wt[i] + (time - tat[i]);
```

```
        tqt = 1;
```

```
        while(tqt <= qt && bt[i] != 0)
```

```
        {
```

```
            lmore = TRUE;
```

```
            bt[i] = bt[i] - 1;
```

```
            tqt++;
```

```
            time++;
```

```
            tat[i] = time;
```

```
        }
```

```
    }
```

```
}
```

```
printf("\nProcessor ID\tBurstTime\tTurnAroundTime\tWaitingTime\n");
```

```
for(i = 0; i < n; i++)
```

```
{
```

```
    printf("Processor%d\t\t%d\t\t%d\t\t%d\n",
```

```
        i + 1, tbt[i], tat[i], wt[i]);
```

```

        t_tat = t_tat + tat[i];
        t_wt = t_wt + wt[i];
    }

    printf("\nTotal Turn Around Time:%d", t_tat);
    printf("\nAverage Turn Around Time:%d", t_tat / n);
    printf("\nTotal Waiting Time:%d", t_wt);
    printf("\nAverage Waiting Time:%d", t_wt / n);
}

```

Output:

```

Enter no. of processors:3
Enter Quantum Time:10
Enter Burst Time of Processor[1]:3
Enter Burst Time of Processor[2]:2
Enter Burst Time of Processor[3]:8

Processor ID    BurstTime    TurnAroundTime    WaitingTime
Processor1      3            3                  0
Processor2      2            5                  3
Processor3      8            13                 5

Total Turn Around Time:21
Average Turn Around Time:7
Total Waiting Time:8
Average Waiting Time:2ghost@LAPTOP-HNSF11IH:~$ █

```


Ex 7a – Process Synchronization Using Semaphores

Program:

```
#include <stdio.h>
```

```
int n = 0, buffersize = 0, currentsize = 0;
```

```
void producer()
```

```
{
```

```
    printf("\nEnter number of elements to be produced: ");
```

```
    scanf("%d", &n);
```

```
    if (0 <= (buffersize - (currentsize + n)))
```

```
    {
```

```
        currentsize += n;
```

```
        printf("%d Elements produced by producer where buffersize is  
%d\n",
```

```
            currentsize, buffersize);
```

```
    }
```

```
    else
```

```
    {
```

```
        printf("\nBuffer is not sufficient\n");
```

```
    }
```

```
}
```

```
void consumer()
```

```
{
```

```
    int x;
```

```
    printf("\nEnter no. of elements to be consumed: ");
```

```
    scanf("%d", &x);
```

```

        if (currentsize >= x)
        {
            currentsize -= x;
            printf("\nNumber of elements consumed: %d, Number of Elements
left: %d",
                x, currentsize);
        }
        else
        {
            printf("\nNumber of Elements consumed should not be greater than
Number of Elements produced\n");
        }
    }
}

```

```

void main()
{
    int c;

    printf("\nEnter maximum size of buffer:");
    scanf("%d", &buffersize);

    do
    {
        printf("\n1.Producer 2.Consumer 3.Exit");
        printf("\nEnter Choice:");
        scanf("%d", &c);

        switch(c)
        {
            case 1:
                if (currentsize >= buffersize)
                    printf("\nBuffer is full. Cannot produce");

```

```
        else
            producer();
        break;

    case 2:
        if (currentsize <= 0)
            printf("\nBuffer is Empty. Cannot consume");
        else
            consumer();
        break;

    case 3:
        break;

    default:
        printf("\nInvalid choice");
    }

} while(c != 3);
}
```

Output:

Enter maximum size of buffer:5

1.Producer 2.Consumer 3.Exit

Enter Choice:1

Enter number of elements to be produced: 6

Buffer is not sufficient

1.Producer 2.Consumer 3.Exit

Enter Choice:1

Enter number of elements to be produced: 5

5 Elements produced by producer where buffersize is 5

1.Producer 2.Consumer 3.Exit

Enter Choice:2

Enter no. of elements to be consumed: 6

Number of Elements consumed should not be greater than Number of Elements produced

1.Producer 2.Consumer 3.Exit

Enter Choice:2

Enter no. of elements to be consumed: 3

Number of elements consumed: 3, Number of Elements left: 2

1.Producer 2.Consumer 3.Exit

Enter Choice:1

Ex 7b – Dining Philosophers Using Semaphores

Program:

```
#include <stdio.h>
#include <semaphore.h>
#include <pthread.h>
#include <unistd.h>

#define N 5
#define THINKING 0
#define HUNGRY 1
#define EATING 2

#define LEFT (ph_num + 4) % N
#define RIGHT (ph_num + 1) % N

sem_t mutex;
sem_t S[N];

void *philospher(void *num);
void take_fork(int);
void put_fork(int);
void test(int);

int state[N];
int phil_num[N] = {0, 1, 2, 3, 4};

int main()
{
    int i;
    pthread_t thread_id[N];
```

```

    sem_init(&mutex, 0, 1);

    for(i = 0; i < N; i++)
        sem_init(&S[i], 0, 0);

    for(i = 0; i < N; i++)
    {
        pthread_create(&thread_id[i], NULL, philospher, &phil_num[i]);
        printf("Philosopher %d is thinking\n", i + 1);
    }

    for(i = 0; i < N; i++)
        pthread_join(thread_id[i], NULL);

    return 0;
}

void *philospher(void *num)
{
    while(1)
    {
        int *i = num;

        sleep(1);
        take_fork(*i);

        sleep(0);
        put_fork(*i);
    }
}

```

```

void take_fork(int ph_num)
{
    sem_wait(&mutex);

    state[ph_num] = HUNGRY;
    printf("Philosopher %d is Hungry\n", ph_num + 1);

    test(ph_num);

    sem_post(&mutex);

    sem_wait(&S[ph_num]);
    sleep(1);
}

void test(int ph_num)
{
    if (state[ph_num] == HUNGRY &&
        state[LEFT] != EATING &&
        state[RIGHT] != EATING)
    {
        state[ph_num] = EATING;

        sleep(2);

        printf("Philosopher %d takes fork %d and %d\n",
            ph_num + 1, LEFT + 1, ph_num + 1);

        printf("Philosopher %d is Eating\n", ph_num + 1);

        sem_post(&S[ph_num]);
    }
}

```

```
}
```

```
void put_fork(int ph_num)
```

```
{
```

```
    sem_wait(&mutex);
```

```
    state[ph_num] = THINKING;
```

```
    printf("Philosopher %d putting fork %d and %d down\n",  
          ph_num + 1, LEFT + 1, ph_num + 1);
```

```
    printf("Philosopher %d is thinking\n", ph_num + 1);
```

```
    test(LEFT);
```

```
    test(RIGHT);
```

```
    sem_post(&mutex);
```

```
}
```


Ex 8 – Banker's Algorithm

Program:

```
#include <stdio.h>
#include <stdlib.h>

void main() {
    int k=0 , output[10], d=0,t=0,ins[5],i,
    avail[5],allocated[10][5], need[10][5], MAX[10][5], pno , P[10] , j , rz,
    count=0;

    system("clear");

    printf("\n Enter the number of resources : ");
    scanf("%d", &rz);
    printf("\n enter the max instances of each resources\n");

    for(i=0;i<rz;i++) {
        avail[i]=0;
        printf("%c= ",(i+97));
        scanf("%d",&ins[i]);
    }

    printf("\n Enter the number of processes : ");
    scanf("%d", &pno);
    printf("\n Enter the allocation matrix \n ");

    for(i=0;i<rz;i++) printf(" %c", (i+97));
    printf("\n");

    for(i=0;i <pno;i++){
        P[i]=i;
        printf("P[%d] ",P[i]);
```

```

        for(j=0;j<rz;j++) {
            scanf("%d",&allocated[i][j]);
            avail[j]+=allocated[i][j];
        }
    }

    printf("\nEnter the MAX matrix \n ");

    for(i=0;i<rz;i++) {
        printf(" %c",(i+97));
        avail[i]=ins[i]-avail[i];
    }

    printf("\n");

    for(i=0;i <pno;i++) {
        printf("P[%d] ",i);
        for(j=0;j<rz;j++) scanf("%d", &MAX[i][j]);
    }

    printf("\n");

    d=-1;

    for(i=0;i <pno;i++) {
        P[i]=i;
        printf("P[%d] ",P[i]);
        for(j=0;j<rz;j++) {
            scanf("%d",&allocated[i][j]);
            avail[j]+=allocated[i][j];
        }
    }

```

```

printf("\nEnter the MAX matrix \n ");

for(i=0;i<rz;i++) {
    printf(" %c",(i+97));
    avail[i]=ins[i]-avail[i];
}

printf("\n");

for(i=0;i <pno;i++) {
    printf("P[%d] ",i);
    for(j=0;j<rz;j++) scanf("%d", &MAX[i][j]);
}

printf("\n");

d=-1;

printf(">");
}

```

Ex 9 – Shell Commands

1. echo \$1 \$2

```
ghost@LAPTOP-HNSF11IH:~$ sh 1.sh Hello World
Hello World
```

2. var1=10
var2=20
sum=\$((var1 + var2))
echo \$sum

```
30
```

3. echo "Enter a Number"
read num

if ! ["\$num" =~ ^-?[0-9]+\$]
then
 echo "Not a valid integer"
elif ["\$num" -lt 0]
then
 echo "Negative"
elif ["\$num" -gt 0]
then
 echo "Positive"
else
 echo "Neither Positive Nor Negative"
fi

```
ghost@LAPTOP-HNSF11IH:~$ bash 1.sh
Enter a Number
10
Positive
```

4. i=1
while [\$i -le 10]
do
 echo \$i
 i=\$((i+1))
done

done

```
ghost@LAPTOP-HNSF11IH:~$ bash 1.sh
1
2
3
4
5
6
7
8
9
10
```

```
5. function add()
{
    sum=$(( $1 + $2 ))
    echo "Sum = $sum"
}
```

```
a=10
b=20
add $a $b
```

```
ghost@LAPTOP-HNSF11IH:~$ bash 1.sh
Sum = 30
```

```
6. read -p "Enter two strings: " str1 str2
```

```
if [ $str1 == $str2 ]
then
    echo "Equal"
else
    echo "Un Equal"
fi
```

```
ghost@LAPTOP-HNSF11IH:~$ bash 1.sh
Enter two strings: Hello World
Un Equal
```